FEB 4- HACKER RANK PROBLEMS

QUESTION 1:

The Pan Am 73 flight from Bombay to New York en route to Karachi and Frankfurt was hijacked by a few Palestinian terrorists at the Karachi International Airport. The senior flight purser Neerja Banhot had to wither her fear and start evacuating the passengers on board. She pleaded the hijackers to release the oldest and the youngest person in the aircraft. Heeding to her plea the chief of the hijacker agreed to let go the oldest and the youngest. Given the ages of the passengers find the oldest and the youngest.

Input Format

The first line of input consists of an integer n, corresponding to the number of passengers in the aircraft. The next line consists of the age of passengers separated by a space.

Constraints

No Constraints

Output Format

The output prints the youngest and oldest separated by a space. Print Invalid Input if n or any one of the ages is negative.

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;


public class Solution {


    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);

        int n=sc.nextInt();
```

```
        int a[]=new int[n],i;
        for(i=0;i<n;i++){
            a[i]=sc.nextInt();
        }
        int min=a[0];
        int max=a[0];
        for(i=0;i<n;i++){
            if(min>a[i]){
                min=a[i];
            }
        }for(i=0;i<n;i++){
            if(max<a[i]){
                max=a[i];
            }
        }
        System.out.println("Youngest="+min);
        System.out.println("Oldest="+max);
    }
}
```

QUESTION 2:

Given the array of integers nums, you will choose two different indices i and j of that array. Return the maximum value of (nums[i]-1)*(nums[j]-1).

Input Format

Input: nums = [3,4,5,2] Output: 12 Explanation: If you choose the indices i=1 and j=2 (indexed from 0), you will get the maximum value, that is, (nums[1]-1)(nums[2]-1) = (4-1)(5-1) = 3*4 = 12. Input: nums = [1,5,4,5] Output: 16 Explanation: Choosing the indices i=1 and j=3 (indexed from 0), you will get the maximum value of (5-1)*(5-1) = 16. Input: nums = [3,7] Output: 12

Constraints

2 <= nums.length <= 500 1 <= nums[i] <= 10^3

Output Format

Execute the Maximum Product Of Two Elements In An Array

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;

public class Solution {

  public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();

      int max1 = Integer.MIN_VALUE;

      int max2 = Integer.MIN_VALUE;

      for (int i = 0; i < n; i++) {

         int x = sc.nextInt();

         if (x > max1) {

            max2 = max1;

            max1 = x;

         } else if (x > max2) {
```

```
            max2 = x;
        }
    }

    int result = (max1 - 1) * (max2 - 1);
    System.out.print("Maximum product of two elements in an array is " + result);
  }
}
```

QUESTION 3:

The function accepts an integer sum and an integer array arr of size n. Implement the function to find the pair, (arr[j], arr[k]) where j!=k, Such that arr[j] and arr[k] are the least two elements of the array (arr[j] + arr[k] <= sum) and return the product of the element of this pair

Input Format

Given in the problem statement

Constraints

No Constraints

Output Format

Given in the problem statement

SOLUTION:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
```

```
import java.util.regex.*;

public class Solution {


    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int sum = sc.nextInt();

        int n = sc.nextInt();



        if (n < 2) {

            System.out.print(-1);

            return;

        }

        int min1 = Integer.MAX_VALUE;

        int min2 = Integer.MAX_VALUE;

        for (int i = 0; i < n; i++) {

            int x = sc.nextInt();

            if (x < min1) {

                min2 = min1;

                min1 = x;

            } else if (x < min2) {

                min2 = x;

            }
```

```
        }
        if (min1 + min2 <= sum) {
            System.out.print(min1 * min2);
        } else {
            System.out.print(0);
        }
    }
}
```

QUESTION 4:

A party has been organised on cruise. The party is organised for a limited time(T). The number of guests entering (E[i]) and leaving (L[i]) the party at every hour is represented as elements of the array. The task is to find the maximum number of guests present on the cruise at any given instance within T hours.

Input Format

Given in the problem statement

Constraints

1<=T<=25 0<= E[i] <=500 0<= L[i] <=500

Output Format

Given in the problem statement

SOLUTION:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
```

```
import java.util.regex.*;


public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        int[] E= new int[T];
        int[] L= new int[T];
        int i;
        for (i = 0; i<T; i++) {
            E[i] = sc.nextInt();
        }
        for (i = 0; i<T; i++) {
            L[i] = sc.nextInt();
        }
        int cu= 0;
        int max = 0;
        for (i = 0;i<T; i++) {
            cu+= E[i]-L[i];
            max= Math.max(max,cu);
        }
        System.out.println("Maximum number of guests within " + T + " hours is " + max);
```

```
}}
```

QUESTION 5:

The function accepts two positive integers ‘r’ and ‘unit’ and a positive integer array ‘arr’ of size ‘n’ as its argument ‘r’ represents the number of rats present in an area, ‘unit’ is the amount of food each rat consumes and each ith element of array ‘arr’ represents the amount of food present in ‘i+1’ house number, where 0 <= i

Input Format

Input:

r: 7 unit: 2 n: 8 arr: 2 8 3 5 7 4 1 2

Constraints

No Constrains

Output Format

Output:

4

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;

   public class Solution {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int r= sc.nextInt();
```

```
int u=sc.nextInt();

int n=sc.nextInt();

int a[]=new int[n];

int i,j,b=0,count=0;

for (i = 0; i <n; i++) {

    a[i] = sc.nextInt();

}

int req=r*u;

for (i = 0; i < n; i++) {

    b+= a[i];

    if (b>= req) {

        System.out.print("Amount of food in 1st " + (i + 1) + " houses is sufficient for all the rats");

        return;

    }

}

System.out.print(0);}}
```

QUESTION 6:

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Input Format

A list of integers representing the elements of the array.

Constraints

The length of the array (n) is at least 1. The majority element always exists in the array.

Output Format

An integer representing the majority element.

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;



public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);



        int n = sc.nextInt();

        int[] a = new int[n];



        for (int i = 0; i < n; i++) {

            a[i] = sc.nextInt();

        }
```

```
        Arrays.sort(a);

        System.out.println(a[n / 2]);

    }

}
```

QUESTION 7:

Choose an index of an array and check such that the sum of elements at previous indexes is equal to the sum of elements at next indexes from that choosen index.

Example : If Input: A[] = {-7, 1, 5, 2, -4, 3, 0} Then Output: 3., 3 is a balanced index, because: A[0] + A[1] + A[2] = A[4] + A[5] + A[6] though A[3] is the choosen one.

Input Format

read an integer n and read n number of integer elements

Constraints

Nil

Output Format

find and print the equilibrium index number or return -1 if not found

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;



public class Solution {
```

```
public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt(),i,j;
    int left=0,right=0;
    int arr[]=new int[n];
    for(i=0;i<n;i++){
        arr[i]=sc.nextInt();
    }

    for(i=0;i<n;i++){
        left=left+arr[i];
        right=0;
        for(j=i+2;j<n;j++){
          right=right+arr[j];}
            if(left==right){
               System.out.print(i+1);
                return;
            }
```

```
        }

        System.out.print(-1);

    }

}
```

QUESTION 8:

Develop a program that reads an array of integers from the user, calculates the frequency of each unique element, and prints the elements alongside their frequencies. The array's size and elements will be provided by the user at runtime.

NOTE:YOU MUST NOT USE ANY ALGORITHM AND INBUILT METHODS FOR THIS QUESTION

Input Format

The first line of input contains a single integer, N, representing the size of the array. The second line contains N space-separated integers representing the elements of the array.

Constraints

No Constraints

Output Format

The output should consist of several lines, each containing an integer and its frequency in the format element:frequency.

SOLUTION:

```
import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;
```

```
public class Solution {


    public static void main(String[] args) {

      Scanner sc=new Scanner(System.in);

       int n=sc.nextInt(),i,j;

       int arr[]=new int[n];

       for(i=0;i<n;i++){

           arr[i]=sc.nextInt();

       }

       int count=1;

        boolean visited[]=new boolean[n];

       for(i=0;i<n;i++){

           count=1;

           if(visited[i]==false){

           for(j=i+1;j<n;j++){

               if(arr[i]==arr[j]){

                   count++;

                   visited[j]=true;

               }

           }

         System.out.println(arr[i]+":"+count);  }

       }
```

```
    }

}
```